



The philosophy of OO programming

Type	Lecture
Date	@January 8, 2022
Lecture #	1
Lecture URL	https://youtu.be/PF6bAB-d9hl
Notion URL	https://21f1003586.notion.site/The-philosophy-of-OO-programming-43af67bf48ea4fd99b7b3a7a488fb8f3
Week #	3

Algorithms + Data Structures = Programs

- Title of Niklaus Wirth's introduction to Pascal
- Traditionally, algorithms come first
- Data representation comes later

Object Oriented design

- Reverses the traditional focus
- First, identify the data we want to maintain and manipulate
- Then, identify algorithms to operate on the data
- **Claim:** Works better for large systems
- **Example:** Simple web browser
 - 2000 procedures maintaining global data
 - ... vs. 100 classes, each with about 20 methods
 - Much easier to grasp the design
 - Debugging: an object is in an incorrect state
 - Search among 20 methods rather than 2000 procedures

Object Oriented design: Example

- An order processing system typically involves

- Items
- Orders
- Shipping addresses
- Payments
- Accounts
- What happens to these objects?
 - Items are *added* to orders
 - Orders are *shipped, cancelled*
 - Payments are *accepted, rejected*
- Nouns signify objects, verbs denote the methods that operate on objects
 - Associate with each order, a method to add an item

Designing objects

- Behaviour — what methods do we need to operate on objects?
- State — how does the object react when methods are invoked?
 - State is the info in the instance variables
- Encapsulation — should not change unless a method operates on it
- Identity — distinguish between different objects of the same class
 - State may be the same — two orders may contain the same item
- These features interact
 - State will typically affect behaviour
 - Cannot add an item to an order that has been shipped
 - Cannot ship an empty order

Relationship between classes

- Dependence
 - `Order` needs `Account` to check credit status
 - `Item` does not depend on `Account`
 - Robust design minimizes dependencies, or coupling between classes
- Aggregation
 - `Order` contains `Item` objects
- Inheritance
 - One object is a specialized version of another
 - `ExpressOrder` inherits from `Order`
 - Extra methods to compute shipping charges, priority handling

Summary

- An object-oriented approach can help organize code in large projects



Subclasses and Inheritance

Type	Lecture
Date	@January 8, 2022
Lecture #	2
Lecture URL	https://youtu.be/6tDGUnETvv4
Notion URL	https://21f1003586.notion.site/Subclasses-and-Inheritance-25f005c285f04a58927eed6577ddb8f2
Week #	3

A typical Java class

```
public class Employee {
    private String name;
    private double salary;

    /* Constructors ... */

    // Mutator methods
    public boolean setName(String s) { ... }
    public boolean setSalary(double x) { ... }

    // Accessor methods
    public String getName() { ... }
    public String getSalary() { ... }

    // Other methods
    public double bonus(float percent) {
        return (percent/100.0) * salary;
    }
}
```

What do we have here

- An `Employee` class
- Two private instance variables
- Some constructors to set up the object
- Accessor and Mutator methods to set instance variables
- A public method to compute bonus

Subclasses

- Managers are special types of employees with extra features

```
public class Manager extends Employee {
    private String secretary;
    public boolean setSecretary(String s) { ... }
    public String getSecretary() { ... }
}
```

- `Manager` object inherit other fields and methods from `Employee`
 - Every `Manager` has a `name`, `salary` and methods to access and manipulate these
- `Manager` is a *subclass* of `Employee`
- `Manager` objects do not automatically have access to private data of the parent class
 - Common to extend a parent class written by someone else
- How can a constructor for `Manager` set instance variables that are private to `Employee`?
 - Use parent class's constructor using `super`

```
public class Employee {
    ...
    public Employee(String n, double s) {
        name = n;
        salary = s;
    }
    public Employee(String n) {
        this(n, 500.00);
    }
}

public class Manager extends Employee {
    ...
    public Manager(String n, double s, String sn) {
        super(n, s);
        secretary = sn;
    }
}
```

Inheritance

- In general, subclass has more features than parent class
 - Subclass inherits instance variables, methods from the parent class
- Every `Manager` is an `Employee`, but every `Employee` is not a `Manager`
- Can use a subclass in place of a superclass

```
Employee e = new Manager(...);
```

- But the following will not work

```
Manager m = new Employee(...);
```

Summary

- A subclass extends a parent class
- Subclass inherits instance variables and methods from the parent class
- Subclass can add more instance variables and methods
 - Can also override methods — later
- Subclasses cannot see private components of parent class
- Use `super` to access the constructor of parent class



Dynamic dispatch and polymorphism

▼ Type	 Lecture
📅 Date	@January 8, 2022
☰ Lecture #	3
🔗 Lecture URL	https://youtu.be/zN-6WryuloU
🔗 Notion URL	https://21f1003586.notion.site/Dynamic-dispatch-and-polymorphism-Qd56d1e9d22a468d9f2fd3744afd0b25
# Week #	3

Subclasses and inheritance

- A subclass extends a parent class
- Subclass inherits instance variables and methods from the parent class
- Subclasses cannot see private components of the parent class
- Subclass can add more instance variables and methods
- Can also override methods

```
public class Employee {
    private String name;
    private double salary;

    public boolean setName(String s) { ... }
    public boolean setSalary(double x) { ... }
    public String getName() { ... }
    public double getSalary() { ... }

    public double bonus(float percent) {
        return (percent/100.0) * salary;
    }
}

public class Manager {
    private String secretary;
    public boolean setSecretary(String s) { ... }
    public String getSecretary() { ... }
}
```

Dynamic dispatch

- `Manager` can redefine `bonus()`

```
double bonus(float percent) {
    return 1.5 * super.bonus(percent);
}
```

- Uses parent class `bonus()` via `super`
- Overrides definition in parent class
- Consider the following assignment
`Employee e = new Manager(...);`
- Can we invoke `e.setSecretary()`?
 - `e` is declared to be an `Employee`
 - Static typechecking — `e` can only refer to methods in `Employee`
- What about `e.bonus(p)`? Which `bonus()` do we use?
 - Static → use `Employee.bonus()`
 - Dynamic → use `Manager.bonus()`
- **Dynamic dispatch** (dynamic binding, late method binding, ...) turns out to be more useful
 - Default in Java, optional in languages like C++ (`virtual` function)

Polymorphism

```
Employee[] emparray = new Employee[2];
Employee e = new Employee(...);
Manager m = new Manager(...);

emparray[0] = e;
emparray[1] = m;

for(int i = 0; i < emparray.length; ++i) {
    System.out.println(emparray[i].bonus(5.0));
}
```

- Every `Employee` in `emparray` “knows” how to calculate its `bonus` correctly
- Recall the event simulation loop that motivated Simula to introduce objects
- Also referred to as runtime polymorphism or inheritance polymorphism

Functions, signatures and overloading

- Signature of a function is its name and the list of argument types
- Can have different functions with the same name and different signatures
 - For example, multiple constructors
- Java class `Arrays` has a method `sort` to sort arbitrary scalar arrays
- Made possible by overloaded methods defined in class `Arrays`

```
double[] darr = new double[100];
int[] iarr = new int[500];
...
Arrays.sort(darr); // Sort the contents of darr
Arrays.sort(iarr); // Sort the contents of iarr

class Arrays {
    ...
    public static void sort(double[] a) { ... } // Sort arrays of double[]
    public static void sort(int[] a) { ... } // Sort arrays of int[]
    ...
}
```

- **Overloading:** multiple methods, same name, different signatures (different parameters), choice is static
- **Overriding:** multiple methods, same name, same signature, choice is static

- `Employee.bonus()`
- `Manager.bonus()`
- **Dynamic dispatch:** multiple methods, same signature, choice made at run-time

Type casting

- Consider the following assignment

```
Employee e = new Manager(...)
```

- Can we get `e.getSecretary()` to work?
 - Static type checking disallows this

- Type casting — convert e to Manager

```
((Manager) e).setSecretary(s)
```

- Cast fails (errors at run time) if `e` is not a `Manager`
- Can test if `e` is a `Manager`

```
if(e instanceof Manager) { ((Manager) e).setSecretary(s); }
```

- We can also use type casting for basic data types

```
double d = 29.98;
```

```
int nd = (int) d;
```

Summary

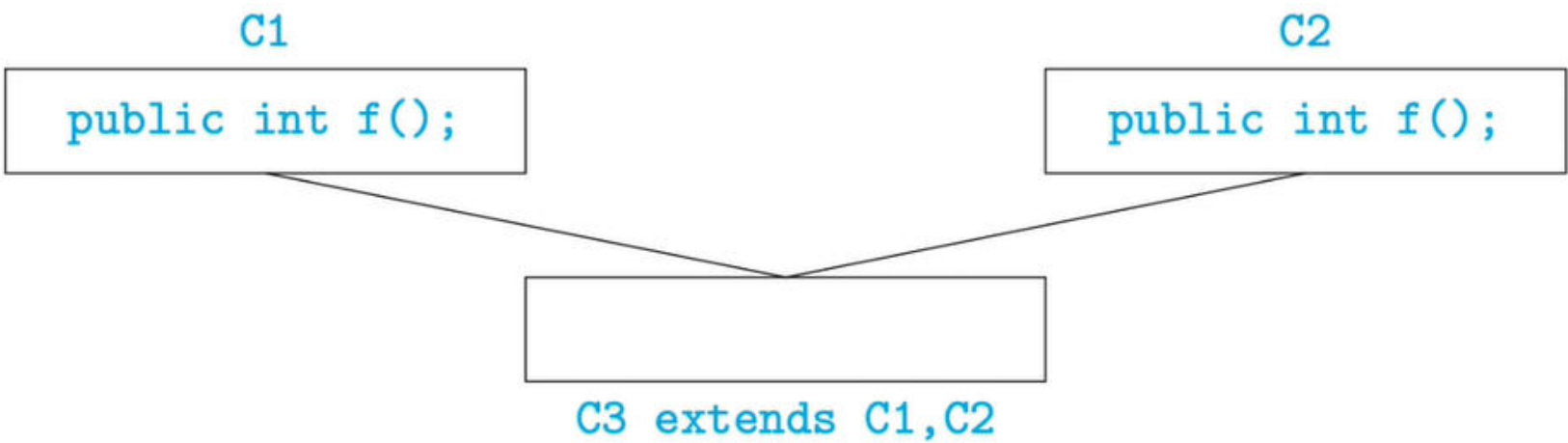
- A sub-class can override a method from a parent class
- Dynamic dispatch ensures that the most appropriate method is called, based on the run-time identity of the object
- Run-time/inheritance polymorphism, different from overloading
 - We will later see another type of polymorphism, *structural polymorphism*
 - For instance, use the same sorting functions for array of any datatype that supports a comparison operation
 - Java uses the term *generics*
- Use type-casting (and reflection) overcome static type restrictions



The Java class hierarchy

Type	Lecture
Date	@January 8, 2022
Lecture #	4
Lecture URL	https://youtu.be/WRN3QWICaag
Notion URL	https://21f1003586.notion.site/The-Java-class-hierarchy-42f67eff075e438a81fe1dc7c31725fa
Week #	3

Multiple inheritance



- Can a sub-class extend multiple parent classes?
- If `f()` is not overridden, which `f()` do we use in `C3` ?
- Java does NOT allow multiple inheritance
- C++ allows this only if `C1` and `C2` have no conflict

Java class hierarchy

- No multiple inheritance — tree-like
- In fact, there is a universal superclass `Object`
- Useful methods defined in `Object`

```
public boolean equals(Object o) // defaults to pointer equality
public String toString() // converts the value of the instance variables to String
```


- For Java objects `x` and `y`, `x == y` invokes `x.equals(y)`
- To print `o`, use `System.out.println(o + "");`
 - Implicitly invokes `o.toString()`
- Can exploit the tree structure to write generic functions
 - Example: search for an element in an array

```
public int find(Object[] objarr, Object o) {
    int i;
    for(i = 0; i < objarr.length(); ++i) {
        if(objarr[i] == o) {
            return i;
        }
    }
    return -1;
}
```

- Recall that `==` is pointer equality, by default
- If a class overrides `equals()`, dynamic dispatch will use the redefined function instead of `Object.equals()` for `objarr[i] == o`

Overriding functions

- For instance, a class `Date` with instance variables `day`, `month` and `year`
- May wish to override `equals()` to compare the object state, as follows

```
public boolean equals(Date d) {
    return this.day == d.day && this.month == d.month && this.year == d.year;
}
```

- Unfortunately, `boolean equals(Date d)` does not override `boolean equals(Object o)`
- Should write this instead

```
public boolean equals(Object d) {
    if(d instanceof Date) {
        Date myd = (Date) d;
        return this.day == myd.day && this.month == myd.month && this.year == myd.year;
    }

    return false;
}
```

- Overriding looks for “closest” match
- Suppose we have `public boolean equals(Employee e)` but no `equals()` in `Manager`
- Consider


```
Manager m1 = new Manager(...);
Manager m2 = new Manager(...);
...
if(m1.equals()m2) { ... }
```
- `public boolean equals(Manager m)` is compatible with both `boolean equals(Employee e)` and `boolean equals(Object o)`
- Use `boolean equals(Employee e)`

Summary

- Java does not allow multiple inheritance
 - A sub-class can only extend one parent class
- The Java class hierarchy forms a tree
- The root of the hierarchy is a built-in class called `Object`
 - `Object` defines default functions like `equals()` and `toString()`
 - These are implicitly inherited by any class that we write

- When we override functions, we should be careful to check the signature



Subtyping vs inheritance

Type	 Lecture
Date	@January 8, 2022
Lecture #	5
Lecture URL	https://youtu.be/CYY7IT-YHVA
Notion URL	https://21f1003586.notion.site/Subtyping-vs-inheritance-74043d6113e04d2faeab33d3706f824a
Week #	3

Subclasses, subtyping and inheritance

- Class hierarchy provides both subtyping and inheritance
- Subtyping
 - Capabilities of the subtype are a superset of the main type
 - If `B` is a subtype of `A`, wherever we require an object of type `A`, we can use an object of type `B`
 - `Employee e = new Manager(...);` is legal
- Inheritance
 - Subtype can re-use code of the main type
 - `B` inherits from `A` if some functions for `B` are written in terms of functions of `A`
 - `Manager.bonus()` uses `Employee.bonus()`

Subtyping vs inheritance

- Recall the following example
 - `queue`, with methods `insert-rear`, `delete-front`
 - `stack`, with methods `insert-front`, `delete-front`
 - `deque`, with methods `insert-front`, `delete-front`, `insert-rear`, `delete-rear`
- Subtyping
 - `deque` has more functionality than `queue` or `stack`
 - `deque` is a subtype of both these types
- Inheritance

- Can suppress two functions in a `deque` and use it as a `queue` or `stack`
- Both `queue` and `stack` inherit from `deque`



Java modifier

▼ Type	 Lecture
📅 Date	@January 8, 2022
☰ Lecture #	6
🔗 Lecture URL	https://youtu.be/IO-K87_QXGs
🔗 Notion URL	https://21f1003586.notion.site/Java-modifier-fab02a734e3b4208b6c190821c5e9a79
# Week #	3

Modifiers in Java

- Java uses many modifiers in declarations, to cover different features of object-oriented programming
- `public` vs `private` to support encapsulation of data
- `static` , for entities defined inside classes that exist without creating objects of the class
- `final` , for values that cannot be changed
- These modifiers can be applied to classes, instance variables and methods
- Let’s look at some examples of situations where different combinations make sense

`public` VS `private`

- Faithful implementation of encapsulation necessitates modifiers `public` and `private`
 - Typically, instance variables are `private`
 - Methods to query (accessor) and update (mutator) the state are `public`
- We also use `private` methods sometimes to do some work that shouldn’t be directly accessible to public
- Example: a `Stack` class
 - Data stored in a private array
 - Public methods to push, pop, query if empty

```
public class Stack {
    private int[] values;
    private int size;
    private int top;

    public void push(int i) {
        ...
    }
}
```

```

}

public int pop() {
    ...
}

public boolean isEmpty() {
    return top == 0;
}
}

```

- `push()` needs to check if the stack has space

```

public class Stack {
    ...
    public void push(int i) {
        if(top < size) {
            values[top] = i;
            top += 1;
        } else {
            // Deal with stack overflow
        }
        ...
    }
    ...
}

```

- Deal gracefully with stack overflow
 - `private` methods invoked from within `push()` to check if stack is full and expand storage

```

public class Stack {
    ...
    public void push(int i) {
        if(stack_full()) {
            extend_stack();
        }
        ... // Usual push operation
    }
    ...
    private boolean stack_full() {
        return top == size;
    }
    private void extend_stack() {
        /* Allocate additional space, reset size etc. */
    }
}

```

Accessor and Mutator

- Public methods to query and update private instance variables
- `Date` class
 - Private instance variables `day, month, year`
 - One public accessor/mutator method per instance variable
- Inconsistent updates are now possible
 - Separately set invalid combinations of `day` and `month`

```

public class Date {
    private int day, month, year;

    public int getDay() { ... }
    public int getMonth() { ... }
    public int getYear() { ... }

    public void setDay(int d) { ... }
    public void setMonth(int m) { ... }
    public void setYear(int y) { ... }
}

```

- Instead, only allow combined update

```
public class Date {
    private int day, month, year;

    public int getDay() { ... }
    public int getMonth() { ... }
    public int getYear() { ... }

    public void setDate(int d, int m, int y) {
        ...
        // Validate d-m-y combinations
    }
}
```

static components

- Use **static** for components that exist without creating objects
 - Library functions, `main(), ...`
 - Useful constants like `Math.PI, Integer.MAX_VALUE`
- These **static** components are also **public**
- We do use **private static** sometimes
- Internal constants for bookkeeping
 - Constructor sets unique id for each order

```
public class Order {
    private static int lastorderid = 0;
    private int orderid;
    ...
    public Order(...) {
        lastorderid++;
        orderid = lastorderid;
    }
}
```

- `lastorderid` is private static field
- Common to all objects in the class
- Concurrent updates need some care

final components

- **final** denotes that a value cannot be updated
- Usually used for constants (**public** and **static** instance variables)
 - `Math.PI, Integer.MAX_VALUE`
- What would **final** mean for a method?
 - Cannot re-define functions at run-time

Summary

- **private** and **public** are natural artefacts of encapsulation
 - Usually, instance variables are **private** and methods are **public**
 - However, **private** methods also make sense
- Modifiers **static** and **final** are orthogonal to **public/private**
- Use **private static** instance variables to maintain bookkeeping information across objects in a class
 - Global serial number, count number of objects created, profile method invocations ...
- Usually **final** is used with instance variables to denote constants
- A **final** method cannot be overridden by a subclass
- We can also have **private** classes